

Stochastic Subgradient Descent for Convex and Lipschitz Functions

1 Algorithm Description

The algorithm iteratively updates the parameters by computing a subgradient using a single randomly sampled data point and taking a step in the negative subgradient direction. It is designed for convex and Lipschitz objective functions.

1.1 Mathematical Formulation

The algorithm is designed to minimize a convex and Lipschitz continuous objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$. The objective function f is assumed to be convex, meaning that for any $\mathbf{w}_1, \mathbf{w}_2 \in \mathbb{R}^n$ and $\lambda \in [0, 1]$, the following holds:

$$f(\lambda \mathbf{w}_1 + (1 - \lambda) \mathbf{w}_2) \leq \lambda f(\mathbf{w}_1) + (1 - \lambda) f(\mathbf{w}_2).$$

Additionally, f is Lipschitz with constant L , implying:

$$|f(\mathbf{w}_1) - f(\mathbf{w}_2)| \leq L \|\mathbf{w}_1 - \mathbf{w}_2\|$$

for all $\mathbf{w}_1, \mathbf{w}_2 \in \mathbb{R}^n$.

The algorithm operates by iteratively updating the parameter vector $\mathbf{w}$ using a subgradient $\mathbf{g}_t$ at each step t . The subgradient is computed using a single randomly sampled data point or, in the deterministic case, directly from the function. The update rule is:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \mathbf{g}_t$$

where:

- $\mathbf{w}_t$ is the parameter vector at iteration t .
- $\eta_t > 0$ is the step size (learning rate) at iteration t .
- $\mathbf{g}_t$ is a subgradient of f at $\mathbf{w}_t$.

1.2 Hyperparameters

- **Step size (η_t):** Determines the magnitude of the update step. It can be constant or adaptive.
- **Minibatch size:** Number of data points used to compute the subgradient. Here, it is 1.

2 Pseudo Code and Dry Run

2.1 Pseudocode

2.2 Dry Run Example

Consider the function $f(w) = (w - 3)^2$.

Initialization:

- $w_0 = 0$

Algorithm 1 Stochastic Subgradient Descent

```
1: Input: Convex function  $f$ , initial parameters  $\mathbf{w}_0$ , step size schedule  $\eta_t$ 
2: Output: Optimized parameters  $\mathbf{w}$ 
3: Initialize  $\mathbf{w} \leftarrow \mathbf{w}_0$ 
4: Set  $t \leftarrow 0$ 
5: while stopping criterion not met do
6:   Sample a data point (or use deterministic function)
7:   Compute subgradient  $\mathbf{g}_t$  at  $\mathbf{w}_t$ 
8:   Update  $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \eta_t \cdot \mathbf{g}_t$ 
9:    $t \leftarrow t + 1$ 
10: end while
11: return  $\mathbf{w}$ 
```

- Step size $\eta = 0.1$

Iteration 1:

- Compute subgradient $g_0 = \frac{d}{dw}(w - 3)^2|_{w=0} = 2(w - 3)|_{w=0} = -6$
- Update $w_1 = w_0 - \eta \cdot g_0 = 0 - 0.1 \cdot (-6) = 0.6$
- Compute objective value $f(w_1) = (0.6 - 3)^2 = 5.76$

Iteration 2:

- Compute subgradient $g_1 = \frac{d}{dw}(w - 3)^2|_{w=0.6} = 2(0.6 - 3) = -4.8$
- Update $w_2 = w_1 - \eta \cdot g_1 = 0.6 - 0.1 \cdot (-4.8) = 1.08$
- Compute objective value $f(w_2) = (1.08 - 3)^2 = 3.6864$

Iteration 3:

- Compute subgradient $g_2 = \frac{d}{dw}(w - 3)^2|_{w=1.08} = 2(1.08 - 3) = -3.84$
- Update $w_3 = w_2 - \eta \cdot g_2 = 1.08 - 0.1 \cdot (-3.84) = 1.464$
- Compute objective value $f(w_3) = (1.464 - 3)^2 = 2.3593$

This dry run demonstrates how the algorithm converges toward the optimal value at $w = 3$ with iterative updates.

3 Convergence Theory

3.1 Assumptions

- **Convexity:** A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is convex if for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ and $\lambda \in [0, 1]$, we have:

$$f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}).$$

- **Lipschitz Continuity:** A function f is L -Lipschitz if there exists a constant $L > 0$ such that for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$:

$$|f(\mathbf{x}) - f(\mathbf{y})| \leq L \|\mathbf{x} - \mathbf{y}\|.$$

- **Subgradients:** For a convex function f , a vector $\mathbf{g}$ is a subgradient at $\mathbf{x}$ if for all $\mathbf{y}$:

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \mathbf{g}^T (\mathbf{y} - \mathbf{x}).$$

- **Parameter Constraints:** The step size or learning rate α_t satisfies $\sum_{t=1}^{\infty} \alpha_t = \infty$ and $\sum_{t=1}^{\infty} \alpha_t^2 < \infty$.

3.2 Main Convergence Theorem

Consider the iterative update rule for a parameter $\mathbf{x}_t$:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha_t \mathbf{g}_t$$

where $\mathbf{g}_t$ is a subgradient of f at $\mathbf{x}_t$. Under the stated assumptions, the following convergence properties hold: If f is a convex Lipschitz function with Lipschitz constant L , and α_t satisfies the parameter constraints, then the sequence $\{\mathbf{x}_t\}$ generated by the algorithm satisfies:

$$\min_{t=1,\dots,T} \mathbb{E}[f(\mathbf{x}_t)] - f^* = O\left(\frac{L^2}{\sqrt{T}}\right)$$

where f^* is the minimum value of the function f .

3.3 Theoretical Properties

- **Stability Analysis:** The choice of α_t ensures stability in the sense that the updates do not diverge. A diminishing step size avoids oscillations and ensures convergence.
- **Sensitivity to Hyperparameters:** The convergence rate is sensitive to the choice of the step size sequence $\{\alpha_t\}$. Proper tuning can significantly affect the practical performance of the algorithm.
- **Comparison to Baseline Methods:** Compared to full gradient methods, the stochastic subgradient method offers computational efficiency by using one data point per update, at the expense of slower convergence rates.

3.4 Discussion

The stochastic subgradient descent method for convex Lipschitz functions provides a robust framework for optimization in high-dimensional spaces, especially where accessing full gradients is computationally prohibitive. While the theoretical convergence rate is slower compared to deterministic methods, the efficiency in handling large-scale data makes it a viable option in practical scenarios. Future work may explore adaptive step sizes and acceleration techniques to enhance convergence properties without sacrificing the stochastic nature of the updates.

4 Convergence Proofs

4.1 Preliminaries (Definitions and Lemmas)

4.1.1 Definitions

1. **Convex Function:** A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is convex if for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ and $\lambda \in [0, 1]$, we have:

$$f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}).$$

2. **Lipschitz Continuity:** A function f is L -Lipschitz if there exists a constant $L > 0$ such that for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$:

$$|f(\mathbf{x}) - f(\mathbf{y})| \leq L \|\mathbf{x} - \mathbf{y}\|.$$

3. **Subgradient:** For a convex function f , a vector $\mathbf{g}$ is a subgradient at $\mathbf{x}$ if for all $\mathbf{y}$:

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \mathbf{g}^T (\mathbf{y} - \mathbf{x}).$$

4. **Parameter Constraints:** The step size or learning rate α_t satisfies $\sum_{t=1}^{\infty} \alpha_t = \infty$ and $\sum_{t=1}^{\infty} \alpha_t^2 < \infty$.

4.1.2 Lemmas

For a convex function f with subgradient $\mathbf{g}_t$ at $\mathbf{x}_t$, the following inequality holds:

$$f(\mathbf{x}_t) - f^* \leq \mathbf{g}_t^T (\mathbf{x}_t - \mathbf{x}^*).$$

Proof. By the definition of subgradients, for any $\mathbf{x}^*$ (in particular, the minimizer of f), we have:

$$f(\mathbf{x}^*) \geq f(\mathbf{x}_t) + \mathbf{g}_t^T (\mathbf{x}^* - \mathbf{x}_t).$$

Rearranging gives:

$$f(\mathbf{x}_t) - f(\mathbf{x}^*) \leq \mathbf{g}_t^T (\mathbf{x}_t - \mathbf{x}^*).$$

□

4.2 Main Proof (Step-by-Step Derivation)

Consider the iterative update rule:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha_t \mathbf{g}_t.$$

We aim to show that:

$$\min_{t=1, \dots, T} \mathbb{E}[f(\mathbf{x}_t)] - f^* = O\left(\frac{L^2}{\sqrt{T}}\right).$$

4.2.1 Step 1: Bounding the Distance

First, consider the squared distance between $\mathbf{x}_{t+1}$ and $\mathbf{x}^*$:

$$\begin{aligned} \|\mathbf{x}_{t+1} - \mathbf{x}^*\|^2 &= \|\mathbf{x}_t - \alpha_t \mathbf{g}_t - \mathbf{x}^*\|^2 \\ &= \|\mathbf{x}_t - \mathbf{x}^*\|^2 - 2\alpha_t \mathbf{g}_t^T (\mathbf{x}_t - \mathbf{x}^*) + \alpha_t^2 \|\mathbf{g}_t\|^2. \end{aligned}$$

4.2.2 Step 2: Using the Subgradient Inequality

Using the subgradient inequality:

$$f(\mathbf{x}_t) - f^* \leq \mathbf{g}_t^T (\mathbf{x}_t - \mathbf{x}^*),$$

we substitute into the distance bound:

$$\|\mathbf{x}_{t+1} - \mathbf{x}^*\|^2 \leq \|\mathbf{x}_t - \mathbf{x}^*\|^2 - 2\alpha_t (f(\mathbf{x}_t) - f^*) + \alpha_t^2 \|\mathbf{g}_t\|^2.$$

4.2.3 Step 3: Taking Expectations

Taking expectations and summing from $t = 1$ to T , we have:

$$\mathbb{E}[\|\mathbf{x}_{T+1} - \mathbf{x}^*\|^2] \leq \|\mathbf{x}_1 - \mathbf{x}^*\|^2 - 2 \sum_{t=1}^T \alpha_t \mathbb{E}[f(\mathbf{x}_t) - f^*] + \sum_{t=1}^T \alpha_t^2 \mathbb{E}[\|\mathbf{g}_t\|^2].$$

Assuming $\|\mathbf{g}_t\| \leq L$, we simplify:

$$\sum_{t=1}^T \alpha_t \mathbb{E}[f(\mathbf{x}_t) - f^*] \leq \frac{\|\mathbf{x}_1 - \mathbf{x}^*\|^2}{2} + \frac{L^2}{2} \sum_{t=1}^T \alpha_t^2.$$

4.2.4 Step 4: Convergence Rate

Dividing by $\sum_{t=1}^T \alpha_t$, we obtain:

$$\min_{t=1, \dots, T} \mathbb{E}[f(\mathbf{x}_t)] - f^* \leq \frac{\|\mathbf{x}_1 - \mathbf{x}^*\|^2}{2 \sum_{t=1}^T \alpha_t} + \frac{L^2}{2} \frac{\sum_{t=1}^T \alpha_t^2}{\sum_{t=1}^T \alpha_t}.$$

Using the parameter constraints, $\sum_{t=1}^T \alpha_t \approx \sqrt{T}$ and $\sum_{t=1}^T \alpha_t^2 \approx 1$, the rate becomes:

$$\min_{t=1, \dots, T} \mathbb{E}[f(\mathbf{x}_t)] - f^* = O\left(\frac{L^2}{\sqrt{T}}\right).$$

4.3 Rate Derivation (With Constants)

To derive the rate with constants, assume $\alpha_t = \frac{1}{\sqrt{t}}$. Then:

$$\sum_{t=1}^T \alpha_t \approx 2\sqrt{T}, \quad \sum_{t=1}^T \alpha_t^2 \approx \log(T).$$

Thus, the rate with constants is:

$$\min_{t=1, \dots, T} \mathbb{E}[f(\mathbf{x}_t)] - f^* \leq \frac{\|\mathbf{x}_1 - \mathbf{x}^*\|^2}{4\sqrt{T}} + \frac{L^2 \log(T)}{4\sqrt{T}}.$$

4.4 Special Cases

1. **Constant Step Size:** If $\alpha_t = \alpha$, the convergence rate becomes linear but requires careful tuning of α .
2. **Adaptive Step Size:** Techniques like AdaGrad can be employed to adaptively adjust α_t , potentially improving practical performance.

4.5 Convergence Proofs Under Different Assumption Settings

The following proofs extend the main convergence result to different assumption settings. For brevity, we outline the key differences and results, assuming familiarity with the main proof.

4.5.1 Proof 1: Strongly Convex Functions

Assumption: The function f is μ -strongly convex, i.e., for all $\mathbf{x}, \mathbf{y}$:

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \mathbf{g}^T(\mathbf{y} - \mathbf{x}) + \frac{\mu}{2} \|\mathbf{y} - \mathbf{x}\|^2.$$

Result: With constant step size $\alpha_t = \frac{1}{\mu t}$, the convergence rate improves to:

$$\mathbb{E}[f(\mathbf{x}_t)] - f^* = O\left(\frac{L^2}{\mu t}\right).$$

Proof Sketch: The strong convexity adds a quadratic term to the subgradient inequality, which accelerates convergence. The step size schedule ensures the distance to the optimum decreases faster.

4.5.2 Proof 2: Non-Smooth Functions

Assumption: The function f is convex and Lipschitz but not necessarily differentiable.

Result: The main proof holds without modification, as the subgradient exists for non-smooth convex functions.

Proof Sketch: The subgradient definition is sufficient to derive the same $O\left(\frac{L^2}{\sqrt{T}}\right)$ rate, as differentiability is not required.

4.5.3 Proof 3: Bounded Domain

Assumption: The parameter space is constrained to a compact convex set $\mathcal{C}$, and updates are projected onto $\mathcal{C}$.

Result: The convergence rate remains $O\left(\frac{L^2}{\sqrt{T}}\right)$.

Proof Sketch: The projection step ensures $\mathbf{x}_t \in \mathcal{C}$, and the boundedness of $\mathcal{C}$ controls the distance terms, preserving the rate.

4.5.4 Proof 4: Noisy Subgradients

Assumption: The subgradient $\mathbf{g}_t$ is noisy, with $\mathbb{E}[\mathbf{g}_t]$ being a valid subgradient and bounded variance.

Result: The rate is $O\left(\frac{L^2 + \sigma^2}{\sqrt{T}}\right)$, where σ^2 is the variance.

Proof Sketch: The noise adds a variance term to the expectation, slightly degrading the rate but maintaining the same order.

4.5.5 Proof 5: Mini-Batch Subgradients

Assumption: The subgradient is computed using a mini-batch of size $m > 1$.

Result: The rate improves to $O\left(\frac{L^2}{\sqrt{mT}}\right)$.

Proof Sketch: The mini-batch reduces the variance of the subgradient estimate, effectively scaling the rate by the batch size.

4.5.6 Proof 6: Adaptive Step Sizes

Assumption: The step size is adapted using methods like AdaGrad.

Result: The rate is $O\left(\frac{L^2}{\sqrt{T}}\right)$, with potentially better constants.

Proof Sketch: Adaptive methods adjust the step size based on past gradients, improving stability and sometimes achieving better practical performance without changing the theoretical rate.